Unit – 4 (continued)

Software Quality Assurance

Cost of Defects • Defect Amplification • SQA Elements & Tasks • Formal & Statistical SQA • Reliability • ISO 9000 • SQA Plan



MCA – 212 • Object-Oriented Software Engineering

Dr Madhavi Dave

Learning Objectives

01 • Cost of Defects

- Explain why the cost of fixing a defect escalates the later it is found.

02 • Defect Amplification

- Model how undetected defects multiply across development phases.

03 • SQA Elements & Tasks

- List the elements, goals, tasks and metrics of a Software Quality Assurance function.

04 • Formal & Statistical SQA

- Contrast formal methods with statistical, Pareto-driven quality assurance.

05 • Reliability & ISO 9000

- Compute MTTF/MTTR/Availability, and place ISO 9000 in an SQA program.

06 • SQA Plan

- Outline the contents of a project's SQA Plan.

SECTION 1

Cost Impact of Software Defects

Why "we'll fix it later" is one of the most expensive sentences in software

Why Defect Cost Matters

The same bug costs wildly different amounts depending on when it's caught

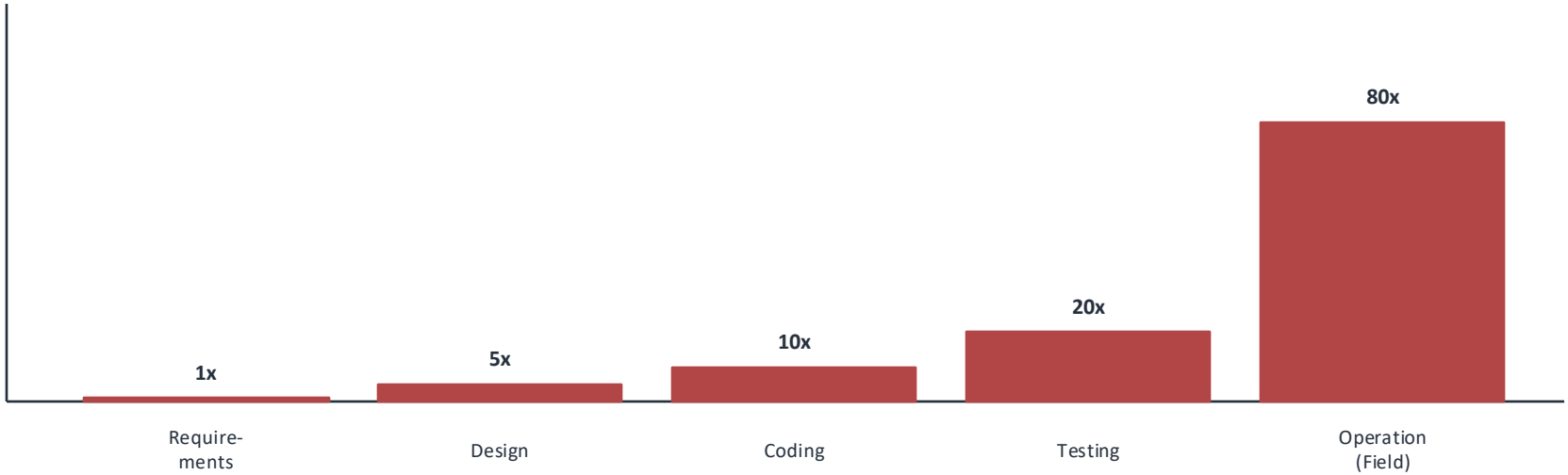
- A defect introduced during requirements analysis is cheap to fix while it is still just a sentence on a page.
- That same conceptual error, if it survives into design, code, and testing, becomes progressively more expensive to trace, isolate and correct.
- If it escapes to production, the cost includes not just the fix, but patch deployment, customer support, and reputational damage.

The core principle

Every stage a defect travels through undetected adds the cost of all the work built on top of the wrong assumption — this is why SQA activities happen throughout the life cycle, not just at the end.

The Rising Cost of Defects

Illustrative relative cost to fix the same defect, by phase found



Reading the chart

These multipliers are illustrative, not universal constants — but the direction is remarkably consistent across studies: catching a defect one phase earlier can cut its cost several times over.

Cost of Quality (COQ) Categories

Every quality dollar falls into one of four buckets

Category	What It Covers	Example
Prevention	Stopping defects before they're created	Training, process improvement, planning
Appraisal	Finding defects that do occur	Reviews, inspections, testing effort
Internal Failure	Fixing defects found before release	Rework, re-testing, debugging
External Failure	Fixing defects found after release	Hotfixes, support calls, recalls, lawsuits

The payoff

Money spent on prevention and appraisal is consistently cheaper, than money spent cleaning up internal and external failures.

Case Study — Cost of One Defect Across Phases

The same missed requirement, priced at each possible discovery point

Discovered During	Illustrative Cost	Why
Requirements review	**$100**	Just a document edit
Design review	**$500**	A diagram and a discussion to redo
Code review / unit test	**$1,500**	Code, plus the tests around it, to rewrite
System test	**$5,000**	Cross-team coordination and re-testing
Production	**$50,000**	Hotfix, support tickets, customer trust

500× is not an exaggeration

The same one-line missing requirement swings from a $100 conversation to a $50,000 incident purely based on when someone noticed it.

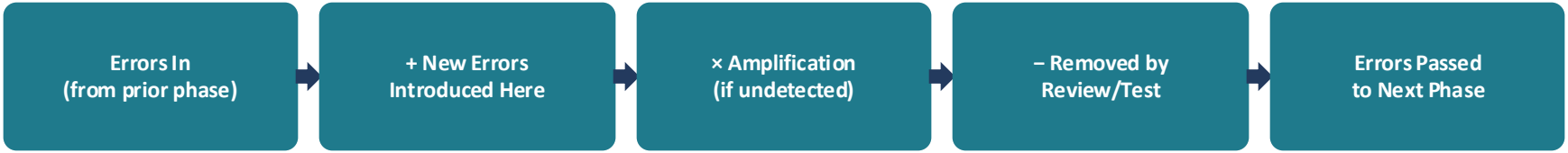
SECTION 2

Defect Amplification and Removal

One missed error rarely stays just one error

The Defect Amplification Model

How an undetected error grows as it moves downstream



- Every development phase can both introduce brand-new errors and amplify errors it inherited — an ambiguous requirement can generate several wrong design decisions, each of which can generate several wrong lines of code.
- A review or test at the end of a phase removes some percentage of the errors present before they are amplified further downstream.
- The fewer errors removed at each gate, the more the surviving errors compound by the time the system reaches production.

Case Study — With Reviews vs. Without Reviews

Same starting point, two very different outcomes (illustrative)

Phase	No Review (errors carried forward)	With Review (60% removed each gate)
Requirements	5	5 → review → 2
Design	10 (×2 amplify)	2×2 + 3 new = 7 → review → 3
Coding	20 (×2 amplify)	3×2 + 4 new = 10 → review → 4
Reaches the Field	**40**	**4**

A 60%-effective review at every phase gate turns a 40-defect field disaster into a 4-defect release — a 10× improvement from the same starting point.

Why Early Reviews Pay Off

Visualizing the amplification effect



The takeaway

This is the mathematical reason walkthroughs, inspections and unit tests are scheduled throughout development rather than saved for one big test phase at the end.

SECTION 3

Elements of Software Quality Assurance

The building blocks of a functioning SQA program

What Is Software Quality Assurance?

A planned, systematic function — not a single test phase

- SQA is the set of planned, systematic activities that provide confidence that a product or service satisfies given quality requirements.
- An SQA group typically sits organizationally independent of the project team, so it can report quality issues honestly without schedule pressure clouding the picture.
- SQA looks at both the product (does the software work?) and the process (was it built the way the organization says it builds software?).

SQA vs. Testing

Testing is one activity inside SQA, not a synonym for it — SQA also covers standards, audits, defect tracking, process compliance and more.

The Elements of SQA

Nine areas a mature SQA program typically addresses

Standards

- Documented conventions the team agrees to follow.

Reviews & Audits

- Technical reviews and process audits at planned checkpoints.

Testing

- Executing the software to find defects before users do.

Defect Collection & Analysis

- Recording defects and mining them for root causes.

Change Management

- Controlling how and when requirements/code change.

Education

- Training staff on quality practices and tools.

Vendor Management

- Ensuring third-party components meet quality bars.

Safety

- Special attention where failure could cause harm.

Risk Management

- Identifying and mitigating quality-related risks.

SECTION 4

SQA Tasks, Goals and Metrics

What the SQA group actually does, and how its own work is measured

SQA Goals

What a Software Quality Assurance function exists to achieve

Improve Quality

- Reduce defects reaching customers through planned, systematic activity.

Ensure Compliance

- Confirm work follows the organization's defined standards and process.

Provide Visibility

- Give management an honest, independent view of product and process health.

Build a Quality Culture

- Make quality everyone's habit, not one team's afterthought.

Independence matters

SQA's value comes largely from its independence — it can raise an uncomfortable finding without its own deadline being on the line.

SQA Tasks

The recurring work of an SQA group across a project



Plan

- Write the SQA Plan; define which reviews, audits and metrics apply to this project.

Review & Audit

- Check that engineering activities and work products follow the defined process.

Track & Escalate

- Document noncompliance and unresolved issues; escalate when necessary.

SQA Metrics

How the SQA function measures its own effectiveness

Metric	What It Tracks
Error/Defect Density	Defects per KLOC or Function Point (from Unit 4 quality metrics)
Defect Removal Efficiency (DRE)	Share of defects caught before release
Review Efficiency	Defects found per hour of review effort
Audit Completion Rate	Planned vs. actually completed audits
Fix Backlog / Aging	Open defects and how long they've been open

SECTION 5

Formal Approaches to SQA

When "probably correct" isn't good enough

Formal Methods for SQA

Mathematics as a quality assurance tool

- Formal methods use mathematical notations and techniques — set theory, logic, formal specification languages — to specify, develop and verify software.
- A formal specification removes the natural-language ambiguity that fuels a huge share of the defects seen in the amplification model earlier in this unit.
- Some formal techniques go further: formal verification mathematically proves that code satisfies its specification, rather than merely testing a sample of behaviors.

Notation / Technique	Typical Use
Z Notation, VDM, B-Method	Formal specification of system behavior
Model checking	Exhaustively verifying state-machine properties

Formal Methods — Benefits & Limitations

A powerful but specialized tool in the SQA toolbox

Benefits

Eliminates entire classes of ambiguity-driven defects at the specification stage.

Can mathematically prove critical properties (e.g. "this train control system never opens two conflicting signals").

Especially valuable where failure is unacceptable — safety- and security-critical systems.

Limitations

Requires specialized mathematical training most teams don't have.

Expensive and slow to scale to large, fast-changing commercial systems.

Proves the specification is met — doesn't guarantee the specification itself is what users actually needed.

SECTION 6

Statistical Software Quality Assurance

Let the defect data tell you where to focus

The Statistical SQA Process

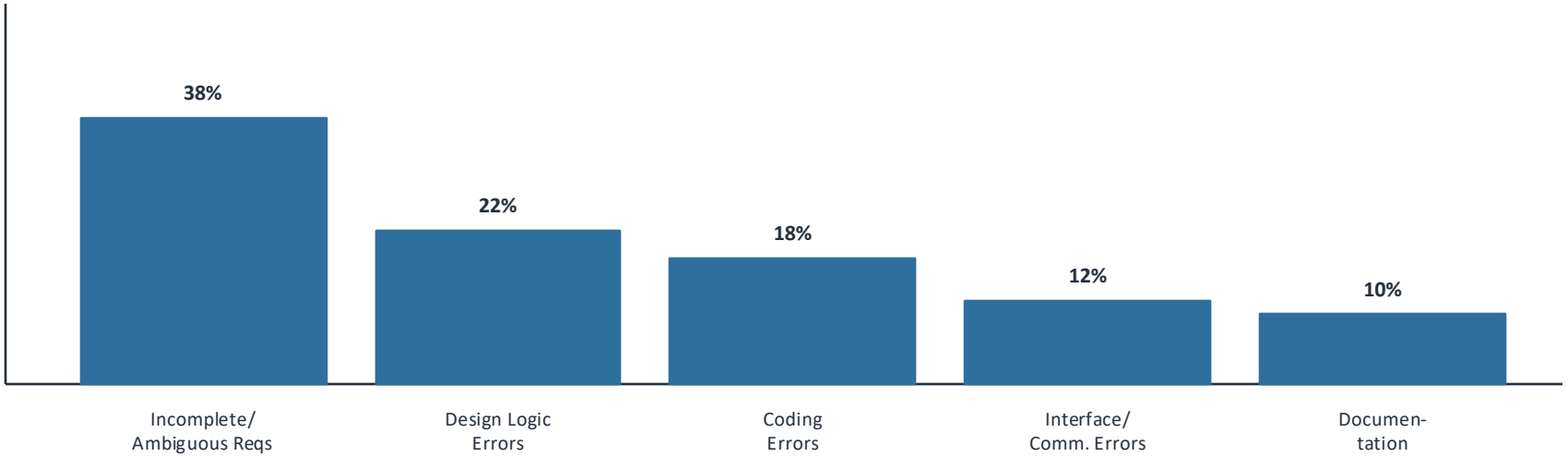
Five steps from raw defect data to a corrected root cause



- Every defect found is logged and categorized (e.g. by the phase and reason it occurred).
- Analysts trace each category back to an underlying cause — often a process gap rather than an individual's mistake.
- The Pareto principle usually holds: roughly 20% of causes account for roughly 80% of defects — fix those first for the best return on effort.

Case Study — Pareto Analysis of Error Causes

Where should this team invest its process-improvement effort?



The vital few

Incomplete/ambiguous requirements and design logic errors together cause 60% of all defects — exactly the two categories a statistical SQA process would target first.

SECTION 7

Software Reliability

Quality, measured the way an operator experiences it

What Is Software Reliability?

The probability of failure-free operation, and how we measure it

- Software reliability is the probability of failure-free operation of a program, for a specified time, in a specified environment.
- Unlike hardware, software doesn't wear out — but latent defects mean reliability is really about how thoroughly a system has been exercised and corrected.

Measure	Meaning
MTTF	Mean Time To Failure — average uptime between failures
MTTR	Mean Time To Repair — average time to restore service
MTBF	Mean Time Between Failures = MTTF + MTTR
Availability	MTTF ÷ (MTTF + MTTR) × 100%

Case Study — Computing Availability

Turning uptime and repair records into one reliability number

Given

Mean Time To Failure (MTTF) = 900 hours

Mean Time To Repair (MTTR) = 100 hours

Computed

MTBF = 900 + 100 = 1,000 hours

Availability = 900 ÷ 1,000 = 90%

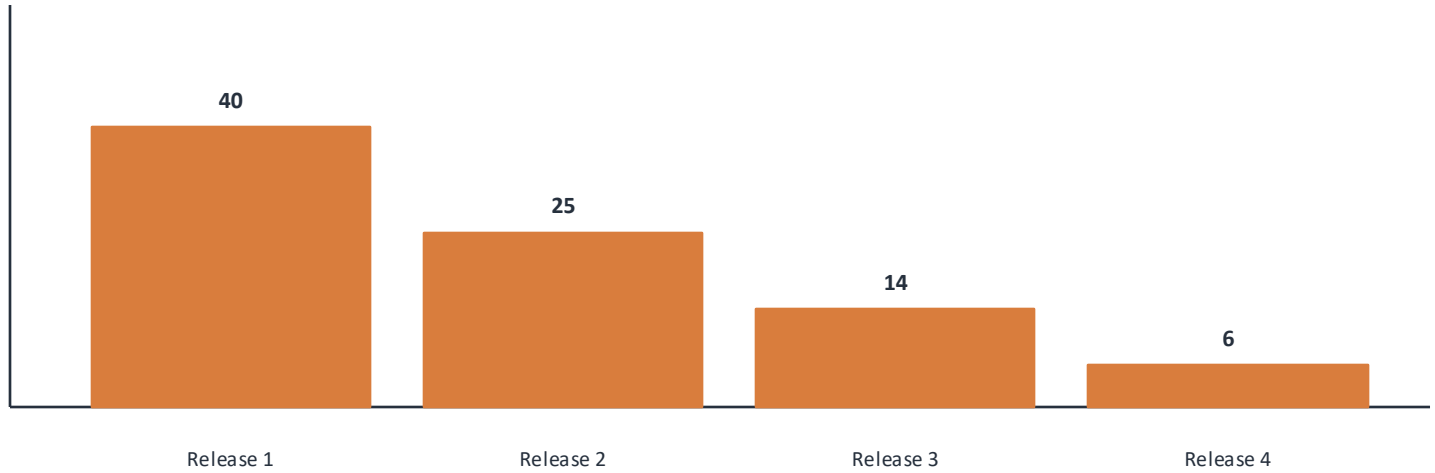
The system is expected to be up and serving requests 90% of the time.

Reading it

90% availability sounds high, but over a year that's still more than 36 days of downtime — reliability targets should always be stated alongside the business impact of that downtime.

Reliability Growth

Reliability improves as defects are found and fixed across releases



Reliability growth models

Models such as Musa's exponential model describe this decline mathematically, letting a team forecast when failure rates will fall below an acceptable release threshold.

SECTION 8

ISO 9000 Quality Standards

An external, internationally recognized definition of "a disciplined process"

The ISO 9000 Family

Three related standards, three different jobs

Standard	Role
ISO 9000	Fundamentals and vocabulary — defines the shared terms and principles
ISO 9001	Requirements — the standard an organization is actually certified against
ISO 9004	Guidance for performance improvement — goes beyond bare compliance

What it does (and doesn't) guarantee

ISO 9001 certifies that an organization follows a consistent, documented, auditable process — it does not directly certify that any particular product is defect-free.

ISO 9001's Seven Quality Management Principles

The philosophy underneath the certification checklist

Customer Focus

- Meet and exceed customer expectations.

Leadership

- Unity of purpose and direction from the top.

Engagement of People

- Competent, empowered people at every level.

Process Approach

- Manage activities as interrelated processes.

Improvement

- Continual improvement as an ongoing objective.

Evidence-Based Decisions

- Decide based on analysis of real data.

Relationship Management

- Manage relationships with suppliers and partners.

What ISO 9001 Certification Means

Earning and keeping the certificate

- An accredited external certification body audits the organization's documented processes and their actual practice.
- Certification is not permanent: periodic surveillance audits check the process is still being followed, not just that it was once written down.
- For a software vendor, certification is often a procurement requirement — many enterprise and government contracts require it before a vendor can even bid.

A common misconception

Being ISO 9001 certified means "we consistently do what we say we do" — it says nothing about whether what they do is actually a good process.

SECTION 9

The SQA Plan

Writing down how quality will actually be assured on this project

Purpose of an SQA Plan

The roadmap that makes SQA activities predictable, not accidental

- An SQA Plan is a project-specific document that defines the organization, tasks, and responsibilities needed to assure software quality.
- It is typically drafted early in project planning, alongside the project management plan, and follows a structure standardized by IEEE Std 730.
- Without a written plan, SQA activities tend to happen inconsistently — heavy at the start, forgotten under deadline pressure.

Who reads it

Project managers use it to schedule reviews and audits; the SQA group uses it as their charter; auditors use it to check whether planned activities actually happened.

Typical SQA Plan Contents

Based on the structure standardized by IEEE Std 730

Section	Covers
Purpose & Scope	What this plan applies to, and why it exists
Management	Organization, tasks, and responsibilities for SQA
Documentation	Which documents govern the project
Standards, Practices & Conventions	The rules the team agrees to follow
Reviews & Audits	What gets reviewed, when, and by whom
Problem Reporting & Corrective Action	How defects are logged, tracked, and closed
Tools, Techniques & Methodologies	What supports the SQA activities
Records Collection & Retention	What evidence is kept, and for how long

Case Study — SQA Plan Excerpt: Library Management System

What a few of these sections might actually say

Reviews & Audits

- Code review every sprint.
- Formal design review before each release.

Problem Reporting

- All defects logged in the tracker within 1 business day.
- Severity triaged within 48 hours.

Metrics Target

- DRE ≥ 90% before release.
- Defect density ≤ 3 / KLOC at ship.

Tools

- Static analysis on every pull request.
- Automated regression suite run nightly.

How It All Connects

From an unnoticed error to a certified, reliable product



- Understanding how defects amplify and get costlier over time is the business case for investing in SQA at all.
- The elements, tasks and metrics of SQA turn that business case into a repeatable, measurable program.
- Formal and statistical methods sharpen where SQA effort goes — mathematical certainty for critical properties, data-driven targeting for everything else.
- Reliability targets, ISO 9000 alignment, and a written SQA Plan tie the whole program to numbers a business — and its customers — can trust.

Key Takeaways

Quality assurance is an investment, not an inspection step

- The cost of a defect rises sharply the later it is caught — this alone justifies investing in reviews long before formal testing begins.
- Left unchecked, a single undetected error can amplify into many downstream errors; reviews at every phase gate break that chain.
- SQA is a defined function with its own goals, tasks and metrics — distinct from, and broader than, testing alone.
- Reliability, ISO 9000 alignment and a documented SQA Plan turn "we care about quality" into something a team can actually plan, measure and prove.